K-Means Clustering Algorithm

Partitioning · Centroids · Inertia · Elbow Method · K-Means++ · Python

AI MASTER CLASS

What We'll Cover Today

01

Clustering Intuition

What is grouping? Why do we need it?

02

The K-Means Algorithm

Step-by-step — Initialize · Assign · Update

03

The Math Deep Dive

Objective function · Convergence proof

04

Full Numerical Example

Run K-Means by hand on real numbers

05

K-Means++ Initialization

Why random init fails & how ++ fixes it

06

Choosing K — Elbow & Silhouette

How to find the right number of clusters

07

Limitations & Variants

Weaknesses, Mini-Batch, Soft K-Means

08

Python Implementation

scikit-learn + full code + visualizations

What is Clustering? — The Big Intuition

Definition: Clustering is the task of grouping data points so that points in the same group are MORE similar to each other than to points in other groups — WITHOUT using any labels.

Customer Segmentation

BEFORE

1 million customers — each has age, income, purchase frequency

AFTER

3 groups emerge: Budget Shoppers · Premium Buyers · Occasional Visitors

💡 Target each group with a different marketing strategy

Gene Expression Analysis

BEFORE

10,000 genes measured across 200 patients — no disease labels

AFTER

4 clusters emerge: subtypes with similar expression profiles

💡 Each cluster may respond differently to treatment

News Article Grouping

BEFORE

50,000 news articles — no categories assigned by humans

AFTER

Topics emerge: Sports · Politics · Tech · Health · Finance

💡 Automatically organize content without any manual tagging

The K-Means Algorithm — Complete Overview

Analogy: Imagine settling into K new cities. Each person moves to whichever city is closest. Cities then relocate to the center of their population. People move again. Repeat until nobody changes cities.

1 — CHOOSE K

Decide how many clusters K you want. This is a hyperparameter — you set it before running. We'll cover how to choose K optimally later.

2 — INITIALIZE

Place K centroids randomly in the feature space. (K-Means++ places them smartly — reduces bad runs dramatically.)

3 — ASSIGN

For every data point x_i, compute distance to each centroid μ_k. Assign x_i to the cluster with the CLOSEST centroid.

4 — UPDATE

Move each centroid μ_k to the mean of all points currently assigned to cluster k: $\mu_k = (1/|C_k|) \cdot \Sigma_{i \in C_k} x_i$

5 — REPEAT 3→4

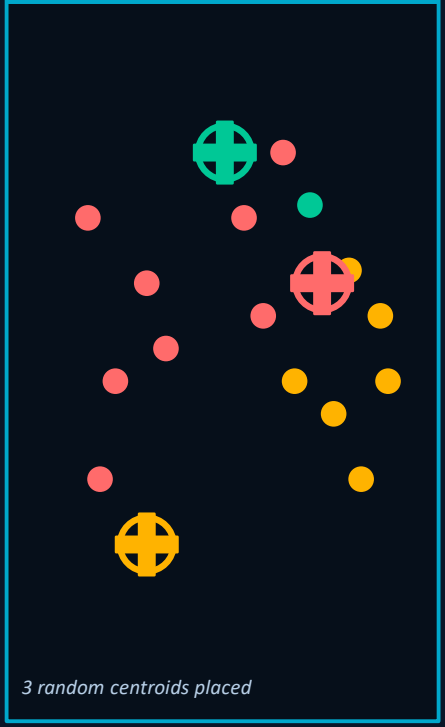
Repeat Assign + Update until no points change clusters (convergence), OR maximum iterations reached.

Assignment step: $c_i = \text{argmin}_k ||x_i - \mu_k||^2$ **Update step:** $\mu_k = (1/|C_k|) \cdot \Sigma_{i \in C_k} x_i$ **Objective:** minimize $J = \Sigma_k \Sigma_{i \in C_k} ||x_i - \mu_k||^2$

K-Means Iterations — Visual Step by Step (K=3)

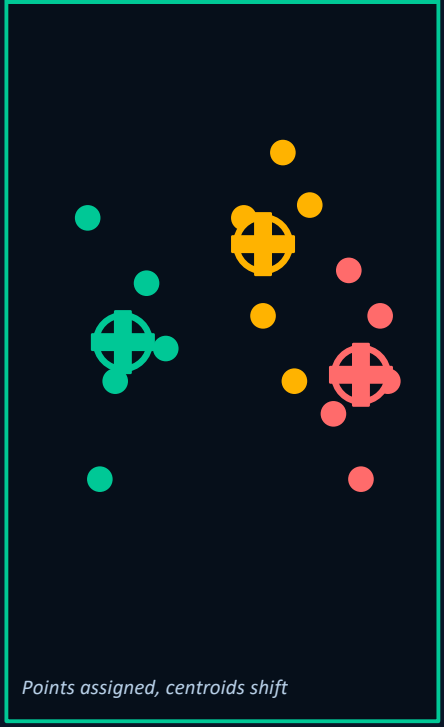
Watch the centroids converge as K-Means runs through 4 iterations on the same dataset.

Iteration 0 — Initialize



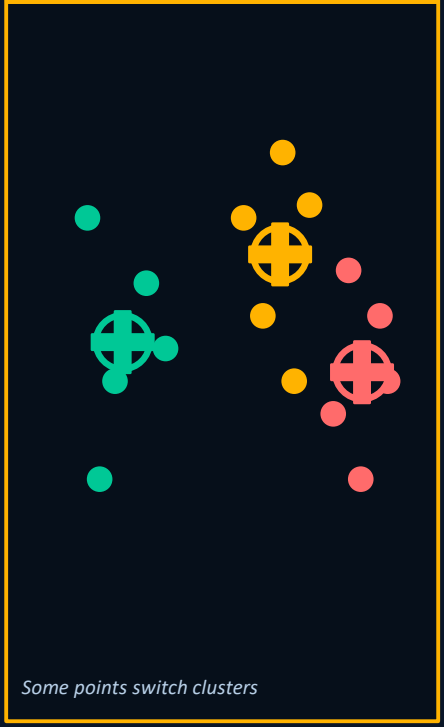
3 random centroids placed

Iteration 1 — Assign + Update



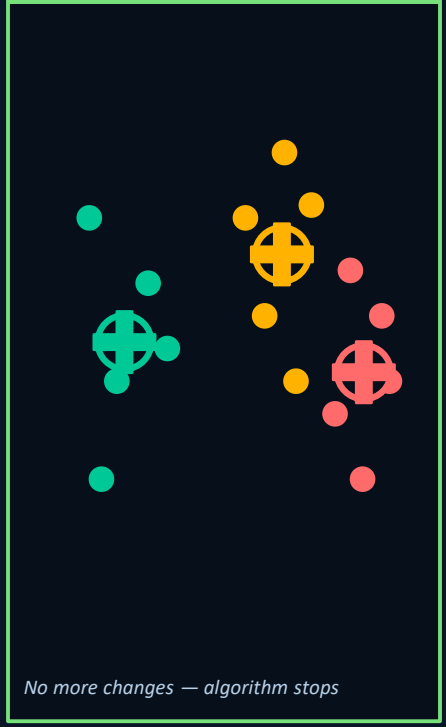
Points assigned, centroids shift

Iteration 2 — Re-assign



Some points switch clusters

Iteration 3 — Converged ✓



No more changes — algorithm stops

● Cluster A ● Cluster B ● Cluster C ✛ Centroid

The Math — Objective Function (Inertia / WCSS)

K-Means minimizes the Within-Cluster Sum of Squares (WCSS), also called inertia.

$$J = \sum_{k=1}^{K} \sum_{i \in C_k} \| x_i - \mu_k \|^2$$

Minimize over all possible assignments $C_1 \ldots C_K$ and all centroids $\mu_1 \ldots \mu_K$

K — **Number of clusters**

The only parameter you set before running. Controls complexity of the model.

C_k — **Set of points in cluster k**

The group of indices i such that data point x_i is assigned to cluster k.

x_i — **Data point i**

A feature vector in $\mathbb{R}^d$ — d-dimensional. Each sample in your dataset.

μ_k — **Centroid of cluster k**

The mean vector of all points in cluster k. Gets updated each iteration.

$\|\cdot\|^2$ — **Squared Euclidean distance**

Sum of squared coordinate differences: $\sum_j (x_{ij} - \mu_{kj})^2$. Always non-negative.

Why K-Means Always Converges — Proof

K-Means is guaranteed to converge in a finite number of steps. Here's why.

Lemma 1 **Assignment step never increases J**

```
J(new assignment) ≤ J(old assignment)
```

Each point is reassigned to the NEAREST centroid. If a point moves to a closer centroid, its squared distance decreases. If it stays, distance is unchanged. So J can only decrease or stay the same.

Lemma 2 **Update step never increases J**

```
The mean minimizes sum of squared distances within a cluster
```

For a set of points $\{x_1 \ldots x_n\}$, the vector that minimizes $\Sigma_i ||x_i - v||^2$ is $v = (1/n)\Sigma_i x_i$ = the mean. Setting μ_k to the mean is optimal — any other center would give higher cost.

Theorem **Finite convergence**

```
J is monotonically non-increasing: J(t+1) ≤ J(t)
```

The number of possible assignments is finite (each of n points has K choices → K^n states). Since J decreases at every step and is bounded below by 0, the algorithm must reach a fixed point (local minimum) in finitely many steps.

⚠ Important: K-Means converges to a LOCAL minimum — not necessarily the global minimum. The result depends on the initial centroid placement. Always run multiple times with different seeds!

Full Numerical Example — Run K-Means by Hand (K=2)

Dataset: 6 points in 2D. We will run K=2 K-Means step by step and compute everything numerically.

Point	x_1	x_2	Init Cluster	Dist to μ_1	Dist to μ_2	After Assign	After Update
A	1	1	μ_1 (K1)	0	5.66	K1	Initial centroids: μ_1 = A = (1, 1) and μ_2 = D = (5, 7) Distance = $\sqrt{[(x-\mu_{1x})^2+(x-\mu_{1y})^2]}$
B	1.5	2	μ_1 (K1)	1.12	4.53	K1	
C	3	4	μ_1 (K1)	3.61	2.83	K2	
D	5	7	μ_2 (K2)	7.81	0	K2	
E	3.5	5	μ_2 (K2)	5.02	1.8	K2	
F	4.5	5	μ_2 (K2)	5.31	1.12	K2	

Updated centroids: $\mu_1(\text{new}) = \text{mean}(A,B) = ((1+1.5)/2, (1+2)/2) = (1.25, 1.5)$ $\mu_2(\text{new}) = \text{mean}(C,D,E,F) = (4.0, 5.25)$

Inertia $= \Sigma\|x_i-\mu_{c_i}\|^2 = \|A-\mu_1\|^2+\|B-\mu_1\|^2+\|C-\mu_2\|^2+\|D-\mu_2\|^2+\|E-\mu_2\|^2+\|F-\mu_2\|^2 = 0.3125+1.0625+5.0+0.5625+0.5625+0.3125 = 7.81$

K-Means++ — Smarter Initialization

Random initialization can give terrible results. K-Means++ fixes this with a probabilistic distance-based seeding.

✗ The Problem with Random Init

All centroids can be initialized in the same cluster → algorithm converges to a bad local minimum with very high inertia. You might need 100+ runs to get a good result.

✓ K-Means++ Solution

Spread centroids far apart from the start. Guarantees initial centroids cover different regions. Provably gives O(log K) competitive approximation ratio to the optimal solution.

1 Pick first centroid

Choose μ_1 uniformly at random from all n data points. No preference — any point is equally likely.

2 Compute distances

For every remaining point x_i, compute $D(x_i)^2$ = min distance² from x_i to the nearest already-chosen centroid.

3 Sample proportionally

Choose next centroid $\mu_?$ with probability proportional to $D(x_i)^2$. Farther points = higher chance of being selected.

4 Repeat until K

Repeat steps 2–3 until all K centroids are chosen. Then run standard K-Means from these smart starting points.

Sampling probability: $P(x_i \text{ chosen as next centroid}) = D(x_i)^2 / \Sigma_j D(x_j)^2$ Code: KMeans(init='k-means++') ← default in scikit-learn!

Choosing K — The Elbow Method

Plot inertia versus K. The 'elbow' where the curve bends gives the optimal K.

Why inertia decreases with K

Adding more clusters always reduces inertia — at K=n (every point is its own cluster), inertia = 0. So we can't just minimize inertia.

What the Elbow tells us

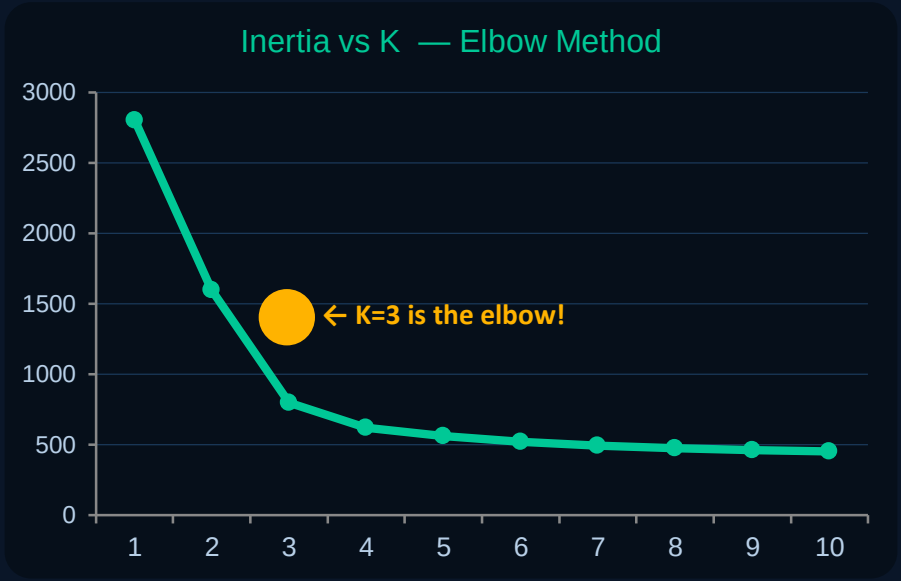
Before the elbow: adding K gives a BIG improvement. After the elbow: adding K gives diminishing returns — you're just splitting natural clusters.

How to find the elbow

Look for the K where the inertia curve sharply bends. Algorithmically: use 'kneedle' algorithm or compute second derivative to find max curvature.

When elbow is ambiguous

Often the curve is smooth with no clear elbow. Then combine with Silhouette Score (next slide) or domain knowledge.



Practical tips for choosing K:

- Try K = 2 to √n (n = sample count)
- Combine Elbow + Silhouette + domain knowledge
- For business use: often K = 3–8 is interpretable
- Gap statistic: compares inertia to random data

Silhouette Score — Measuring Cluster Quality

A score from −1 to +1 for each point: how well does it fit in its own cluster vs the nearest other cluster?

$$s(i) = [\, b(i) - a(i) \,] \;/\; \max(\, a(i) \,,\, b(i) \,)$$

where $a(i)$ = *avg distance from* x_i *to all other points in its own cluster* $b(i)$ = *avg distance from* x_i *to all points in the NEAREST other cluster*

s(i) ≈ +1

Well clustered

Point is deep inside its own cluster and far from all other clusters. Ideal — this is what you want.

s(i) ≈ 0

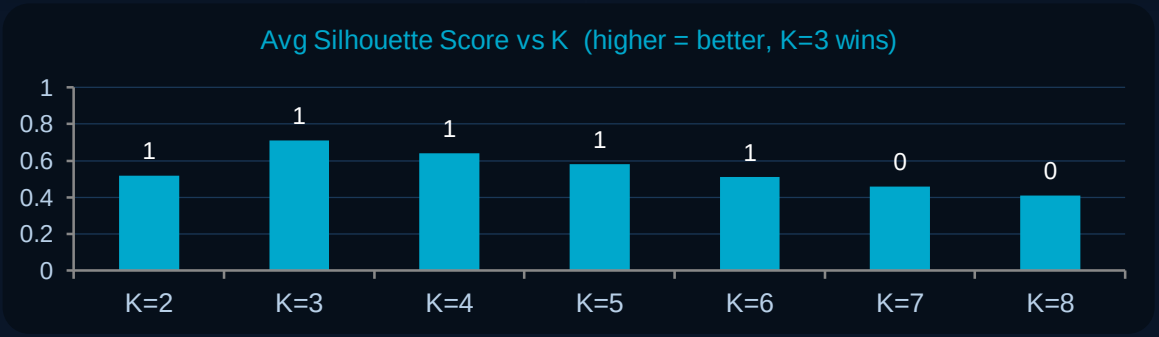
On the boundary

Point is equally close to its own cluster and the next-nearest cluster. Could belong to either.

s(i) ≈ −1

Misclassified

Point is closer to a DIFFERENT cluster than its own. Indicates poor clustering — may need different K or algorithm.



Average Silhouette

Mean of s(i) over all n points. Good clustering: >0.5. Excellent: >0.7.

Best K = highest avg s

Pick K that maximizes the mean Silhouette Score. More reliable than Elbow alone.

K-Means Limitations — When It Fails

K-Means has strong assumptions. Understanding them prevents incorrect use.

Assumes Spherical Clusters

K-Means uses Euclidean distance — it can only find roughly circular (hyperspherical) clusters. Elongated, crescent-shaped, or ring clusters will be split incorrectly.

Fix: Use DBSCAN or Spectral Clustering for non-convex shapes.

Requires Feature Scaling

K-Means is distance-based. A feature with range 0–100,000 completely dominates one with range 0–1. All clusters will be determined by the large-scale feature only.

Fix: Always apply StandardScaler or MinMaxScaler before K-Means.

Must Specify K in Advance

You have to guess the number of clusters before running. Wrong K gives meaningless clusters. There's no automatic K — you must use Elbow + Silhouette to decide.

Fix: Use Elbow + Silhouette. Or use DBSCAN (no K needed).

Sensitive to Initialization

Different random seeds give different results. Algorithm may converge to poor local minima. Standard practice: run with n_init=10 seeds, keep best result.

Fix: Use K-Means++ initialization (default in scikit-learn).

Sensitive to Outliers

One extreme outlier can pull a centroid far away from its true cluster center, ruining the cluster. The mean is not robust to outliers.

Fix: Remove outliers first (Isolation Forest), or use K-Medoids (uses medians, not means).

Assumes Equal-Size Clusters

K-Means tends to create equal-size clusters. If your true clusters are very different in size or density, K-Means will split the large one and merge small ones.

Fix: Use Gaussian Mixture Models (soft assignment with varying covariance).

K-Means Variants — Beyond the Classic Algorithm

Mini-Batch K-Means

Uses small random batches of data (e.g. 1000 points) instead of the full dataset for each update step. Much faster on large datasets — often 2–10× speedup.

When: Best when: n > 100,000 samples. Slight accuracy tradeoff for huge speed gain.

```
MiniBatchKMeans(n_clusters=3, batch_size=1000)
```

K-Medoids (PAM)

Instead of the mean centroid, each cluster center must be an ACTUAL data point (the medoid). Much more robust to outliers than standard K-Means.

When: Best when: Data has outliers. Also works on non-Euclidean distances (categorical data).

```
from sklearn_extra.cluster import KMedoids
```

Soft / Fuzzy K-Means

Each point belongs to each cluster with a DEGREE of membership (0 to 1). Points near borders get split membership. Implemented as Gaussian Mixture Models in sklearn.

When: Best when: Data has overlapping natural clusters. Provides probability of belonging.

```
GaussianMixture(n_components=3, covariance_type='full')
```

Bisecting K-Means

Start with 1 cluster (all data). Repeatedly pick the cluster with highest inertia and split it into 2 using K-Means. Repeat until K clusters. Hierarchical approach.

When: Best when: K is large. More stable results than standard K-Means. Good for large K.

```
BisectingKMeans(n_clusters=10)
```

Spherical K-Means

Uses cosine similarity instead of Euclidean distance. Data points are normalized to the unit sphere. Ideal for text data where direction matters more than magnitude.

When: Best when: TF-IDF vectors, word embeddings, any unit-normalized high-D data.

```
Uses cosine distance: normalize first, then KMeans
```

Kernel K-Means

Maps data to a high-dimensional feature space via kernel trick, then runs K-Means there. Can discover non-linear cluster boundaries like circles and crescents.

When: Best when: Non-convex clusters. But expensive $O(n^2)$. For small datasets only.

```
SpectralClustering(n_clusters=3, affinity='rbf')
```

K-Means in Python — Complete Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.datasets import make_blobs

# 1. Generate / load data
X, y_true = make_blobs(n_samples=500, centers=4, cluster_std=0.8, random_state=42)

# 2. ALWAYS scale features before K-Means!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Find optimal K using Elbow + Silhouette
inertias, sil_scores = [], []
K_range = range(2, 11)
for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, km.labels_))

# 4. Train final model
best_k = 4  # from elbow + silhouette analysis
model = KMeans(n_clusters=best_k, init='k-means++',
               n_init=10, max_iter=300, random_state=42)
labels = model.fit_predict(X_scaled)

# 5. Evaluate
print(f'Inertia       : {model.inertia_:.2f}')
print(f'Silhouette    : {silhouette_score(X_scaled, labels):.3f}')
print(f'Davies-Bouldin: {davies_bouldin_score(X_scaled, labels):.3f}')
```

`n_init=10`

Run 10 times with different seeds, keep the best result. Critical for avoiding bad local minima.

`init='k-means++'`

Always use K-Means++ (the default). Smarter init → faster convergence → better results.

`StandardScaler first!`

K-Means is distance-based. Unscaled features will completely break clustering. Non-negotiable.

`cluster_centers_`

The final centroid positions in scaled space. Use inverse_transform to get original units back.

K-Means Real-World Applications

Customer Segmentation

Group customers by RFM (Recency, Frequency, Monetary value) to personalize marketing. Each cluster gets a different campaign strategy.

Typical K=4–8

Image Compression

Represent each image pixel by its nearest cluster centroid color. K=16 colors can compress a 24-bit image by up to 50% with minimal quality loss.

Typical K=16–256

Document Clustering

Group news articles or research papers by topic. Feature vectors = TF-IDF. Each cluster becomes a 'topic'. Used in search engines and recommendation.

Typical K=10–100

Genomics / Bioinformatics

Group patients or genes by expression profiles → discover disease subtypes. Critical in precision medicine and drug discovery pipelines.

Typical K=2–20

Geographic Segmentation

Cluster GPS locations (stores, accidents, demand hotspots) for logistics optimization — place warehouses at cluster centroids to minimize delivery distance.

Typical K=5–50

Cyber Attack Profiling

Cluster network connection logs into 'normal behavior' groups. Connections that don't fit any cluster well become anomaly candidates for security review.

Typical K=3–10

Session Summary — K-Means Key Takeaways

01

K-Means partitions by proximity

Assign each point to its nearest centroid. Recompute centroids as cluster means. Repeat. Converges to a local minimum of inertia WCSS.

02

Algorithm always converges

J is monotonically non-increasing — both the assign step and update step can only reduce inertia. Finite assignments → finite convergence.

03

Use K-Means++ initialization

Never use pure random init. K-Means++ seeds centroids by distance probability → O(log K) competitive guarantee. Default in sklearn.

04

Scale features — always

Distance-based: StandardScaler is mandatory. Unscaled features completely dominate distance calculations and destroy clusters.

05

Choose K with Elbow + Silhouette

Elbow: plot inertia vs K, find the bend. Silhouette: maximize avg s(i). Combine both + domain knowledge for best K.

06

Know the limitations

Spherical clusters only · Sensitive to outliers · Needs K in advance · Local minimum. Use DBSCAN / GMM when these assumptions fail.

Great work!

You now master K-Means Clustering.